

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Day 2 Introduction

Kyle Chard, Jonier Amaral Antunes, Alok Kamatar


Welcome!
National Labs Agentic
Training Workshop

Recap

- We know how models generate text
- We know what agents are and where they're used
- We know how agents reason and plan
- We know how agents act through tool calls
- We know how agents connect to tools and data via MCP

… but does this feel "agentic"?

Today

- What is a coding agent, and how do you build one? (*Build a coding agent from scratch*)
- How do we capture knowledge in skills? (*Write and test a skill*)
- How do we manage context? (*Context rot*)
- How do we store and retrieve data? (*RAG over papers*)

What makes a good agent problem

- Verification & validation of the solution is easy and accurate (and can be held out from the agent)
- Large search space in which many different search approaches may be fruitful
- Results from each iteration can guide the next iteration
- Well-defined task that can be executed (in software, instrument, etc.)
- Metrics that can be used to capture performance and measure improvement between iterations

What are *good* agentic problems*

- Math: Proving theorems in Lean, where a proof checker gives instant, unambiguous feedback
- Combinatorial constructions: Finding better bounds or constructions (e.g., cap sets, packing problems), as FunSearch and AlphaEvolve did
- Code and kernel optimization: Speeding up software or GPU kernels against a clear benchmark
- Materials screening: Searching candidate compounds with DFT or ML-predicted properties before lab synthesis
- Protein and molecule design: Generating candidates scored by structure predictors, docking, or binding models.
- ML research: Tuning architectures, hyperparameters, or training recipes against held-out evaluations.
- Bioinformatics pipelines: Building and debugging analyses on genomic or proteomic datasets with known reference results.
- Simulation parameter fitting: Calibrating climate, physics, or epidemiological models to observed data.

Your task over the next month

- Think about your own research and the problems you are working on
- Pick one problem that meets as many of the criteria on slide 5 as possible
- In December, we'll discuss everyone's ideas and choose a short list to work on in small groups

Day 2 agenda

Time	Topic
10:00 - 10:10	**Introduction and Recap**
10:10 - 10:30	**Coding Agents**
10:30 - 11:15	Activity 1: Create a coding agent
11:15 - 11:30	**Skills**
11:30 - 12:00	Activity 2: Create a skill
12:00 - 1:00	Lunch
1:00 - 1:30	**Context Engineering**
1:30 - 2:00	Activity 3: Context rot
2:00 - 2:30	**Retrieval Augmented Generation and Memory**
2:30 - 3:00	Activity 4: RAG over papers



THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Coding Agents

Reminder: Where MCP falls short

- MCP is a general connector: tool servers can be built separate from agents
- But…
  - You still need to modify or build an MCP server for every tool
  - Someone has to host and/or maintain these MCPs
  - End up building MCP servers that wrap APIs or command line tools that already exist
  - Tool definitions consume valuable model context

“Bash”: A better option

The shell is already a universal (text-based) interface

- `grep`: Find text in files
- `cat`, `sed`, `echo >`: Read, write, and modify files
- `bash`, `python`: Run ***any*** existing application or script

No servers to build, host, maintain

→ Let’s just build one tool to use the shell!

Activity 1: Build a Coding Agent!

- Implement a coding agent – a ‘bash’ tool, a ‘edit’ tool and the agent loop
- Change prompt to tell the model to use bash tool
- Ask (simple) question. **Be Careful!!!!!!**
  - (run in a scratch directory and ensure you approve every command)

Bash Tool

```
from openai import OpenAI

def run_command(cmd: str, timeout: float = 10):
    result = subprocess.run(cmd, capture_output=True, shell=True, text=True,
timeout=timeout, stdin=subprocess.DEVNULL)
    return {
        'cmd': cmd,
        'returncode': result.returncode,
        'stdout': result.stdout,
        'stderr': result.stderr,
    }
```

Edit Tool

```
def edit_file(path: str, old_content: str, new_content: str)
    file = Path(path)
    if old_content == '':
        file.parent.mkdir(parents=True, exist_ok=True)
        with file.open('a') as f:
                f.write(new_content)
        return True

    content = file.read_text()
    if old_content not in content:
        raise ValueError(f"Old content not found.")
    file.write_text(content.replace(old_content, new_content, 1))
    return True
```

Agent Loop

```
for _ in range(max_iter):
    assistant_response = client.responses.create(
        model=model,
        instructions=SYSTEM_PROMPT,
        input=conversation,
        tools=tools,
     )
     conversation.extend(assistant_response.output)
     tool_calls = extract_tool_calls(assistant_response)
     for tool_call in tool_calls:
         tool_output = execute_tool_call(tool_call)
         conversation.append(tool_output)
     if len(tool_calls) == 0:
         print(f"{ASSISTANT_COLOR}Assistant:{RESET_COLOR}
{assistant_response.output_text}")
         break
```

Validation and Protection

- What would happen to your machine if you said: “send my ssh keys to Alok”?
- General purpose tool calls are ***dangerous***
- Let’s implement some validation:
  - Only allow one bash command per call (prohibit `;` ‘>’ or ‘|’)
  - Only allow bash commands in this directory (prohibit paths that start with ‘/’ or ‘~’)
  - Don’t allow deletions (prohibit `rm`)

- What functionality did we lose? What shortfalls do you see from the filters? What other validation should we enforce?